# 파일코인 전체 아키텍처

## 개요

파일코인 증명 서브시스템(Filecoin Proving Subsystem, FPS)은 파일코인 프로토콜에서 요구하는 저장 증명을 제공하는 핵심 시스템입니다. 전체 시스템은 Rust로 구현되어 있으며, 상호 의존적인 여러 크레이트(crate)로 구성되어 있습니다. 일부 크레이트는 C 바인딩을 통해 외부 API를 제공합니다.

### 아키텍처 설계 원칙

1. **모듈화**: 각 크레이트는 명확한 책임 분리를 통해 독립적으로 개발 및 테스트 가능
2. **재사용성**: 공통 기능은 `storage-proofs-core` 에 집중하여 중복 제거
3. **확장성**: 새로운 증명 타입이나 최적화 기법을 쉽게 추가할 수 있는 구조
4. **성능**: Rust의 제로 코스트 추상화와 메모리 안전성을 활용한 고성능 구현

## 크레이트별 상세 분석

### 1. storage-proofs-core

**역할**: 모든 저장 증명 크레이트에서 사용하는 공통 기초 요소 제공

**핵심 구성 요소**:

**암호학적 기초 ( `crypto/` )**

- 해시 함수 인터페이스 및 구현체
- 암호학적 유틸리티 함수들
- 보안 랜덤 생성기

### 머클 트리 ( `merkle/` )

- Binary, Quad, Oct-ary 머클 트리 구현
- 디스크 기반 트리 저장소 ( `DiskStore` )
- 레벨 캐시 트리 저장소 ( `LevelCacheStore` )
- 머클 증명 생성 및 검증

### DRG (Depth Robust Graph) ( `drgraph.rs` )

- 깊이 견고 그래프의 기본 구현
- 그래프 노드 간 연결성 관리
- 부모 노드 생성 알고리즘

### PoR (Proof of Retrievability) ( `por.rs` )

- 검색 가능성 증명의 기본 구현
- 머클 포함 증명 기반

### 기타 핵심 모듈

- `compound_proof.rs` : 복합 증명 시스템
- `parameter_cache.rs` : 증명 매개변수 캐싱
- `partitions.rs` : 섹터 파티션 관리
- `sector.rs` : 섹터 데이터 구조 및 관리

**의존성**:

- `filecoin-hashers` : 해시 함수 구현체
- `merkletree` : 머클 트리 라이브러리
- `bellperson` : zk-SNARK 회로 구현
- `blstrs` : BLS12-381 타원곡선 연산

## 2. storage-proofs-porep

**역할**: Proof-of-Replication (복제 증명) 구현

**핵심 구성 요소**:

**Stacked DRG 구현 ( `stacked/` )**

- **Vanilla 구현** ( `vanilla/` ): 순수 Rust 구현체
- 레이어별 인코딩 알고리즘
- 부모 노드 캐싱 시스템
- Feistel 암호를 이용한 확장 부모 생성
- **Circuit 구현** ( `circuit/` ): zk-SNARK 회로 구현
- 증명 생성을 위한 회로 제약 조건
- 검증 가능한 계산 로직

**인코딩 모듈 ( `encode.rs` )**

- 데이터 인코딩 및 디코딩 함수
- 레이블 생성 알고리즘
- 레이어 간 데이터 변환

**의존성**:

- `storage-proofs-core` : 기본 암호학적 기초 요소
- `filecoin-hashers` : Poseidon, SHA256 해시 함수
- `sha2raw` : 최적화된 SHA2 구현
- `neptune` : Poseidon 해시 최적화 구현

## 3. storage-proofs-post

**역할**: Proof-of-Space-time (시공간 증명) 구현

**핵심 구성 요소**:

**Fallback PoSt ( `fallback/` )**

- **Vanilla 구현** ( `vanilla.rs` ): 기본 PoSt 알고리즘
- **Circuit 구현** ( `circuit.rs` ): zk-SNARK 회로
- **Compound 구현** ( `compound.rs` ): 복합 증명 래퍼
- **유틸리티** ( `utils.rs` ): 보조 함수들

**주요 기능**

- **Winning PoSt**: 블록 생성 권한 증명
- **Window PoSt**: 지속적인 저장 증명
- **Challenge 생성**: 무작위 도전 생성 및 관리
- **증명 검증**: 제출된 증명의 유효성 검증

**의존성**:

- `storage-proofs-core` : 기본 증명 인프라
- `filecoin-hashers` : 해시 함수 구현

## 4. filecoin-proofs

**역할**: 파일코인 특화 매개변수와 고수준 API 제공

**핵심 구성 요소**:

**API 모듈 ( `api/` )**

- **Sealing API** ( `seal.rs` ): 데이터 봉인 과정
- `seal_pre_commit_phase1` : 1단계 사전 커밋

- `seal_pre_commit_phase2` : 2단계 사전 커밋
- `seal_commit_phase1` : 1단계 커밋
- `seal_commit_phase2` : 2단계 커밋
- **PoSt API** ( `winning_post.rs` , `window_post.rs` ): 시공간 증명
- `generate_winning_post` : Winning PoSt 생성
- `generate_window_post` : Window PoSt 생성
- **유틸리티** ( `util.rs` ): 보조 함수들
- **업데이트** ( `update.rs` ): 섹터 업데이트 기능

**상수 및 매개변수 ( `constants.rs` , `parameters.rs` )**

- 섹터 크기별 매개변수 정의
- 증명 타입별 설정값
- 파일코인 네트워크 특화 상수

**캐시 관리 ( `caches.rs` )**

- 매개변수 파일 캐싱
- 증명 생성 중간 결과 캐싱
- 디스크 기반 캐시 관리

**의존성**:

- `storage-proofs-core` : 기본 증명 시스템
- `storage-proofs-porep` : PoRep 구현
- `storage-proofs-post` : PoSt 구현
- `storage-proofs-update` : 섹터 업데이트 기능

# 보조 크레이트

### filecoin-hashers

- 파일코인에서 사용하는 해시 함수들의 통합 인터페이스
- SHA256, Blake2s, Poseidon 해시 함수 구현
- GPU 가속 지원 (CUDA, OpenCL)

### fr32

- 32바이트 필드 요소 처리
- 패딩 및 언패딩 기능
- 파일코인 데이터 형식 변환

### fil-proofs-param

- 증명 매개변수 관리
- 매개변수 다운로드 및 검증
- 버전 관리

## 데이터 흐름

### Sealing 과정

1. **데이터 준비**: 원본 데이터를 32바이트 청크로 분할
2. **PreCommit Phase 1**:

- Stacked DRG를 이용한 레이어별 인코딩
- 각 레이어에서 부모 노드 기반 레이블 생성

3. **PreCommit Phase 2**:

- TreeC (Column Tree) 구축
- TreeRLast (마지막 레이어 트리) 구축

4. **Commit Phase 1**: 증명 생성을 위한 데이터 준비
5. **Commit Phase 2**: zk-SNARK 증명 생성

### PoSt 과정

1. **Challenge 생성**: 블록체인에서 무작위 도전 생성
2. **증명 생성**: 저장된 데이터에 대한 머클 증명 생성
3. **회로 실행**: zk-SNARK 회로를 통한 증명 생성
4. **검증**: 생성된 증명의 유효성 검증

## 성능 최적화

### GPU 가속

- CUDA 및 OpenCL 지원
- 머클 트리 구축 가속화
- 해시 계산 병렬화

### 멀티코어 처리

- `multicore-sdr` 기능을 통한 병렬 SDR 처리
- `hwloc` 라이브러리를 이용한 CPU 토폴로지 최적화
- `rayon` 을 이용한 데이터 병렬 처리

### 메모리 최적화

- 부모 캐시 시스템으로 메모리 사용량 최적화

- 디스크 기반 머클 트리로 대용량 데이터 처리
- 메모리 맵 파일을 이용한 효율적인 I/O

# 보안 고려사항

## 암호학적 보안

- BLS12-381 타원곡선 기반 zk-SNARK
- Poseidon 해시 함수의 SNARK 친화적 설계
- 시간-공간 트레이드오프 공격 방어

## 구현 보안

- Rust의 메모리 안전성 보장
- 상수 시간 암호 연산
- 사이드 채널 공격 방어

# 확장성 및 유지보수성

## 모듈러 설계

- 각 증명 타입의 독립적 구현
- 인터페이스 기반 추상화
- 플러그인 방식의 해시 함수 지원

## 버전 관리

- 매개변수 버전 호환성 관리
- API 버전 관리

- 하위 호환성 보장

이 아키텍처는 파일코인의 복잡한 암호학적 요구사항을 만족하면서도, 높은 성능과 확장성을 제공하도록 설계되었습니다. 각 크레이트의 명확한 역할 분리를 통해 개발 및 유지보수의 효율성을 극대화하고 있습니다.